

Atividade Prática Avaliativa: Extensão da "Missão Marte Unifor" em Java (POO)

Disciplina: Projeto e Arquitetura de Sistemas | Curso: Ciência da Computação / Engenharia da Computação / ADS — UNIFOR

- **Modalidade:** Trabalho em Equipe (2 a 3 integrantes)
- **Entrega:** Repositório no GitHub
- **Peso/Nota:** [Inserir Pontuação, ex: 0 a 10,0 pontos na N2]

1. Apresentação e Objetivos

Nesta atividade avaliativa, sua equipe receberá a base de código do jogo em console "**Missão Marte Unifor**" pronta e compilável. O objetivo do grupo é estender a aplicação desenvolvendo os Exercícios Práticos propostos no roteiro da disciplina.

O tutorial completo e o código base para iniciar a atividade estão disponíveis no repositório oficial da disciplina:

 [Tutorial e Código Base - Missão Marte Unifor](#)

A atividade visa consolidar na prática os conceitos de **Herança, Polimorfismo, Encapsulamento, Composição e Manipulação de Arquivos JSON** em Java através da evolução de um sistema existente.

2. Diretrizes do Trabalho em Equipe e Git

- **Formação das Equipes:** Grupos compostos por 2 a 3 alunos.
- **Uso do Git e Distribuição do Trabalho:**
 - O grupo deve acessar o link fornecido acima e clonar a estrutura de pastas do projeto base.
 - A equipe deve criar seu próprio repositório (pode ser um fork ou um repositório novo contendo a base).
 - Todos os alunos devem realizar commits relevantes a partir de suas próprias contas no Git.
- **Histórico de Commits:** Será avaliada a distribuição das tarefas entre os membros através do histórico do repositório.
- **Submissão no Moodle:** Apenas UM integrante do grupo realiza o envio na Tarefa do AVA anexando:
 - Link do Repositório Git da Equipe (público ou com acesso liberado ao professor).
 - Relatório Simplificado em PDF (Relatorio_Equipe.pdf).

3. Escopo do Trabalho: Exercícios Práticos a Desenvolver

A equipe deve implementar as funcionalidades descritas nos exercícios do tutorial (Níveis 1 a 4), integrando-as harmonicamente ao código-fonte original:

Nível 1: Refatoração e Tipos (Exercícios 1 a 3)

- **Ajuste de Capacidade:** Configurar a capacidade da nave para comportar a nova carga de passageiros do sistema.
- **Nova Subclasse de Passageiro (Astronauta):** Criar a classe Astronauta estendendo Passageiro, definindo suas especificidades e adicionando-o ao fluxo do jogo.
- **Customização Visual:** Atualizar a renderização do mapa em console alterando a representação simbólica de elementos (ex: Nave e Asteroides).

Nível 2: Mecânicas de Jogo e POO (Exercícios 4 a 6)

- **Pontuação Polimórfica:** Aplicar sobrescrita de métodos (@Override) para que cada tipo de passageiro ofereça uma pontuação distinta ao ser embarcado (Professor: +10, Engenheiro: +15, Astronauta: +20).
- **Sistema de Vidas na Nave:** Implementar o atributo de vidas na classe Nave e a lógica de perda de vidas ao colidir com asteroides.
- **Mapa Configurável:** Permitir que o jogador informe a dimensão do mapa no início da execução.

Nível 3: Comportamentos Avançados e Persistência (Exercícios 7 a 9)

- **Inimigos Dinâmicos com IA Simples:** Criar a entidade Inimigo com movimentação aleatória pelo mapa a cada turno e verificação de colisão.
- **Menu de Dificuldades (Enum):** Criar o enum Dificuldade (FACIL, MEDIO, DIFICIL) ajustando proporcionalmente recursos, pontuação inicial e quantidade de obstáculos.
- **Persistência Expandida:** Ampliar o formato do ranking.json para salvar e carregar dados adicionais da partida (ex: data/hora, passageiros resgatados e nível de dificuldade).

Nível 4: Desafio Final (Exercício 10)

- **Plataforma de Pouso (0, 0):** Alterar a condição de vitória do jogo. Após resgatar todos os passageiros, o piloto precisa navegar até a Plataforma de Pouso L localizada na coordenada (0, 0) para concluir a missão com sucesso.
- **Menu Principal e Reset:** Implementar opções de menu interativo no início do programa e a funcionalidade de resetar o arquivo do ranking.

4. Orientações de Entrega

Repositório Git:

- **Arquivo README.md:** Localizado na raiz com o nome do grupo, matrículas, link do repositório e instruções de compilação/execução (javac/java).
- **Arquivo .gitignore:** Configurado para não subir os arquivos .class ou pastas de compilação (bin/, out/).
- **Estrutura do Código:** Código-fonte organizado na estrutura de pacotes original (src/missao/).

Relatório PDF (máximo 2 páginas):

- **Cabeçalho:** Link do repositório da equipe no cabeçalho.
- **Tabela de Contribuições:** Tabela indicando o nome dos integrantes, username no Git e os exercícios/funcionalidades desenvolvidos por cada um.
- **Evidências de Execução:** Prints da execução demonstrando as novas funcionalidades em uso.

5. Critérios de Avaliação e Rubrica

A nota da equipe (0 a 10,0) será atribuída de acordo com a extensão e qualidade da implementação dos exercícios sobre o código base:

Critério	Descrição	Pontuação Máxima
1. Resolução dos Exercícios Nível 1 e 2	Implementação da subclasse Astronauta, customização de símbolos do mapa, capacidade da nave, sistema de vidas e mapa reconfigurável.	2,5 pontos
2. Aplicação de POO e Polimorfismo	Uso de sobrescrita @Override para pontuações diferenciadas, criação do enum Dificuldade e correta reutilização de código sem quebrar a arquitetura existente.	2,5 pontos
3. Exercícios Avançados e Regra do Pousa	Implementação da classe Inimigo com movimentação aleatória e alteração da condição de vitória com acoplamento na plataforma (0,0).	2,5 pontos
4. Persistência de Dados e Menu	Menu principal com opções (jogar, ranking, resetar, sair) e extensão do arquivo ranking.json para armazenar mais métricas da partida.	1,5 pontos

Critério	Descrição	Pontuação Máxima
5. Trabalho em Equipe, Git e Documentação	Commits bem distribuídos entre os membros no repositório, presença do arquivo README.md, .gitignore e relatório com o mapeamento de contribuições.	1,0 ponto
TOTAL	Avaliação geral do projeto	10,0 pontos



Observações Importantes

- **Integridade do Código Base:** O código fornecido através do link do professor não deve ter sua lógica principal destruída; os exercícios devem estender e aprimorar as classes existentes.
- **Código Não Compilável:** Projetos que não compilarem após as alterações da equipe sofrerão penalização substancial na nota.
- **Plágio:** Códigos idênticos entre grupos distintos resultarão em nota zero (0,0) para os envolvidos.